

Ex.No :1

Date:

ANALYZE THE ROAD WAY TRAVELS E-R DIAGRAMS

Let us consider the entities

Bus

Reservation

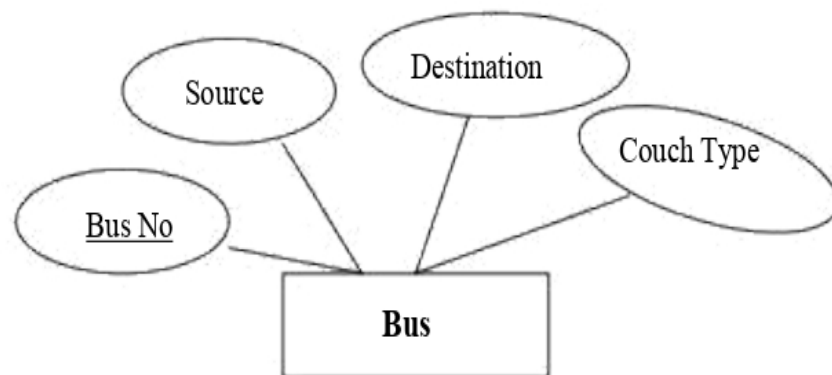
Ticket

Passenger

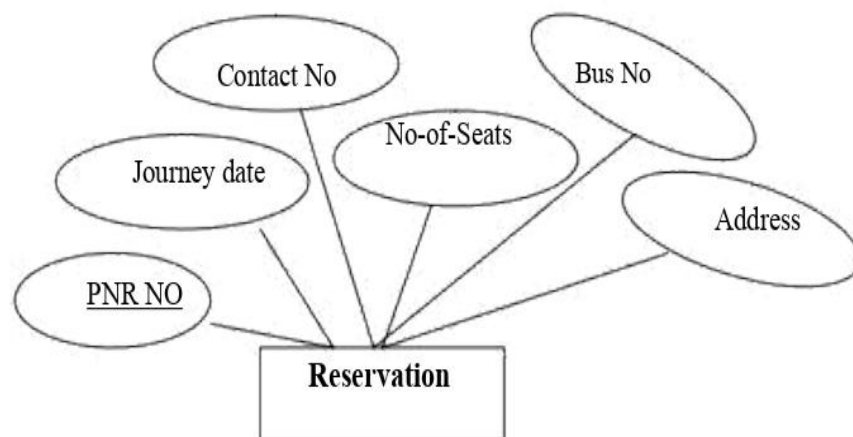
Cancellation

The Attributes in the Entities :

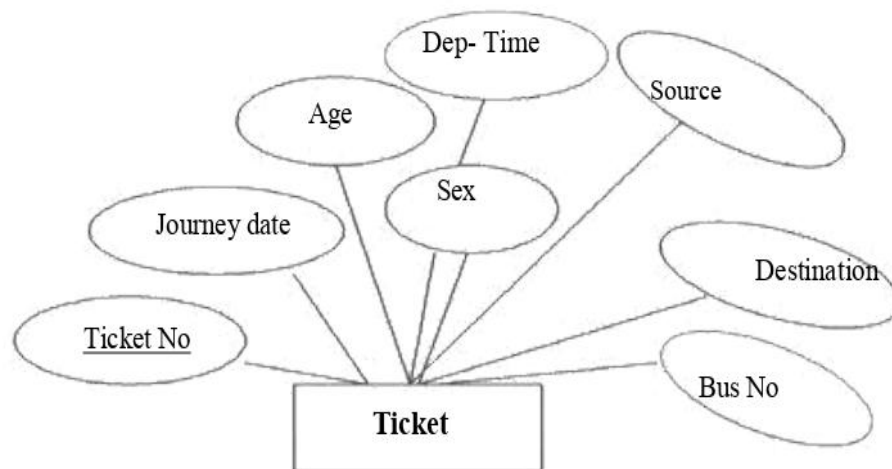
Bus (Entity)



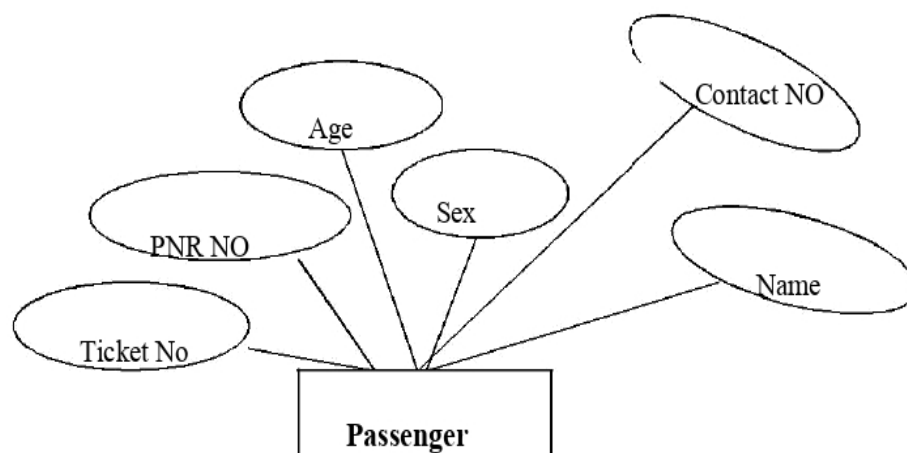
Reservation (Entity)



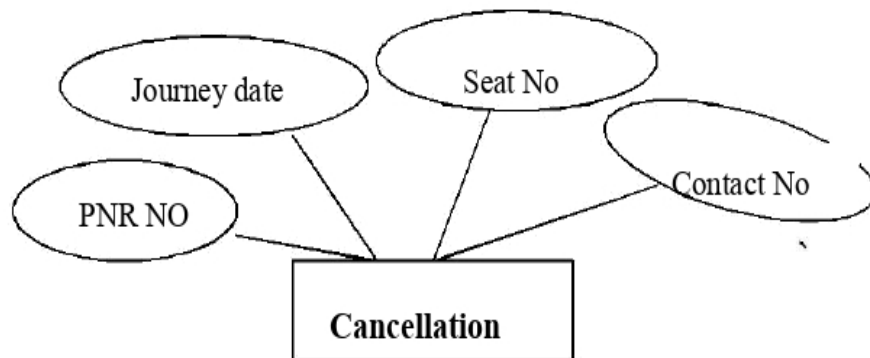
Ticket (Entity)



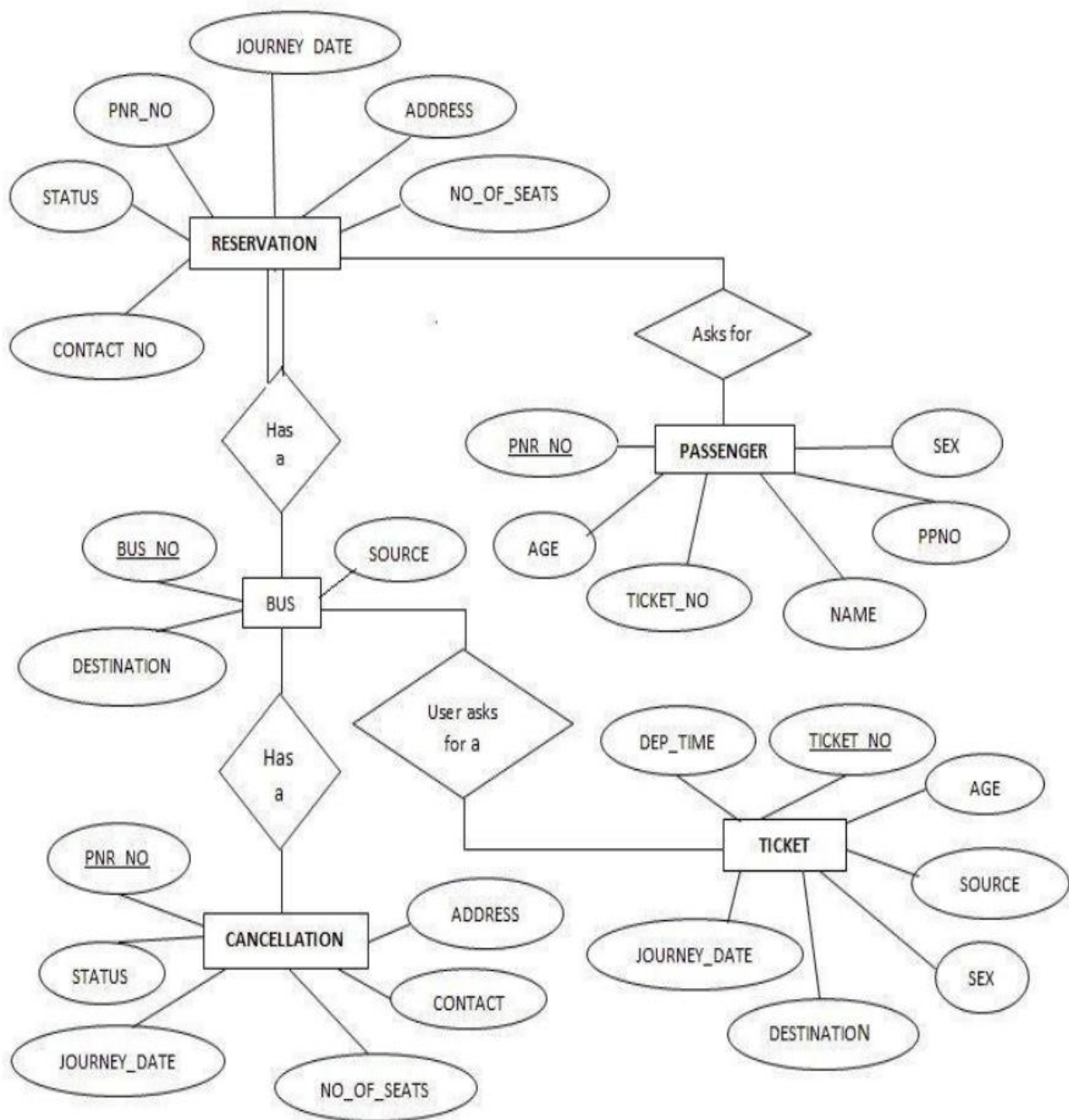
Passenger (Entity)



Cancellation (Entity)



Concept Design with E-R Model



PAAVAI ENGINEERING COLLEGE (Autonomous)		
Description	Marks	Awarded
Preparation & Conduction	10	
Observation & Results	20	
Record Completion	05	
Viva Voce	05	
Total	40	

Result

Thus, the above program has been executed successfully.

Ex.No :2	CREATION OF A DATABASE AND WRITING SQL QUERIES TO RETRIEVE INFORMATION FROM THE DATABASE.
Date:	

Aim:

Creating a **database** and writing **SQL queries** to retrieve information is an essential skill when working with relational databases. Here, I'll walk you through the **creation of a database**, how to **define tables** in the database, and **writing SQL queries** to retrieve and manipulate data.

Algorithm :

Step 1 : Create a Database

Step 2 : Create Tables

Step 3 : Insert Sample Data

Step 4 : Write SQL Queries to Retrieve Information

Step 5 : Stop the process

Program :

i. Create a Database

first need to create a database where you can store your data. Here's an example of how to create a simple database in SQL:

```
CREATE DATABASE Company;
```

This command creates a database named Company.

ii. Create Tables

Once you have the database, you need to create tables to store information. Here's an example where we create two tables: Employees and Departments.

```
USE Company;
```

```
CREATE TABLE Departments (department_id INT PRIMARY KEY, department_name  
VARCHAR(100) NOT NULL);
```

```
CREATE TABLE Employees (  
    employee_id INT PRIMARY KEY, first_name VARCHAR(50), last_name VARCHAR(50),  
    email VARCHAR(100) UNIQUE NOT NULL, hire_date DATE,  
    department_id INT,  
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)  
);
```

iii. Insert Data into Tables

```
INSERT INTO Departments (department_id, department_name) VALUES (1, 'HR'),  
(2, 'Engineering'), (3, 'Marketing');
```

```
INSERT INTO Employees (employee_id, first_name, last_name, email, hire_date,  
department_id) VALUES  
(1, 'Alice', 'Johnson', 'alice.johnson@company.com', '2021-05-01', 1),  
(2, 'Bob', 'Smith', 'bob.smith@company.com', '2020-06-15', 2),  
(3, 'Charlie', 'Brown', 'charlie.brown@company.com', '2022-07-20', 3);
```

iv. Write SQL Queries to Retrieve Information

Basic Query (Retrieve all employees)

```
SELECT * FROM Employees;
```

Filter Data (Retrieve employees from a specific department)

```
SELECT first_name, last_name FROM Employees WHERE department_id = 2;
```

Join Tables (Retrieve employee names and their department names)

```
SELECT e.first_name, e.last_name, d.department_name FROM Employees e JOIN Departments  
d ON e.department_id = d.department_id;
```

Aggregate Functions (Count employees per department)

```
SELECT d.department_name, COUNT(e.employee_id) AS employee_count  
FROM Departments d LEFT JOIN Employees e ON d.department_id = e.department_id  
GROUP BY d.department_name;
```

v. Modify the Database (Update/Delete Data)

You can also modify data using UPDATE and DELETE statements.

Update Data

```
UPDATE Employees SET email = 'alice.johnson@newemail.com'  
WHERE employee_id = 1;
```

Delete Data

```
DELETE FROM Employees WHERE employee_id = 1;
```

PAAVAI ENGINEERING COLLEGE (Autonomous)		
Description	Marks	Awarded
Preparation & Conduction	10	
Observation & Results	20	
Record Completion	05	
Viva Voce	05	
Total	40	

Result

Thus, the above program has been executed successfully.

Ex.No :3	PERFORMING INSERTION, DELETION, MODIFYING, ALTERING, UPDATING AND VIEWING RECORDS BASED ON CONDITIONS.
Date:	

Aim :

The operations for **insertion, deletion, modifying, altering, updating, and viewing records** in an Oracle database

Algorithm :

Step 1 : Inserting records into a table is done using the INSERT INTO statement.

Step 2 : To delete records, use the DELETE statement. You can delete records based on conditions.

Step 3 : To Modify the structure of a table using the ALTER TABLE statement. This can involve adding columns, removing columns, or changing column definitions.

Step 4 : The UPDATE statement is used to modify existing records. You can update specific records based on conditions.

Step 5 : You can view records using the SELECT statement. You can also add conditions to filter the results

Step 6 : you can use WHERE clauses to filter records.

Program :

i. Insertion (INSERT)

```
INSERT INTO Employees (employee_id, first_name, last_name, email, hire_date,  
department_id) VALUES (4, 'David', 'Wilson', 'david.wilson@company.com', '2023-08-10', 2);
```

ii Deletion (DELETE)

```
DELETE FROM Employees WHERE employee_id = 2;
```

iii. Modifying (ALTER TABLE)

The ALTER TABLE command is used to modify the structure of a table, like adding/removing columns or changing data types.

Example i : Adding a new column to the table

```
ALTER TABLE Employees ADD phone_number VARCHAR(20);
```

Example ii : Changing a column's data type

```
ALTER TABLE Employees MODIFY phone_number VARCHAR(50);
```

Example iii : Renaming a column

```
ALTER TABLE Employees RENAME COLUMN phone_number TO contact_number;
```

Example iv : Dropping a column

```
ALTER TABLE Employees DROP COLUMN contact_number;
```

iv. Updating (UPDATE)

```
UPDATE Employees SET email = 'charlie.brown@newemail.com' WHERE employee_id = 3;
```

v. Viewing Records (SELECT)

Example i : View employees from a specific department

```
SELECT * FROM Employees WHERE department_id = 2;
```

Example ii : View employees hired after a specific date

```
SELECT * FROM Employees WHERE hire_date > '2021-01-01';
```

Example iii : View employees whose first name starts with a letter 'A'

```
SELECT * FROM Employees WHERE first_name LIKE 'A%';
```

Example iv : View employees and their department names (using JOIN)

```
SELECT e.first_name, e.last_name, d.department_name FROM Employees e  
JOIN Departments d ON e.department_id = d.department_id;
```

vi. Advanced Conditions (WHERE, AND, OR, IN, BETWEEN)

Example i : Using AND to combine conditions

```
SELECT * FROM Employees WHERE department_id = 1 AND hire_date > '2022-01-01';
```

Example ii : Using OR to fetch records matching either condition

```
SELECT * FROM Employees WHERE department_id = 1 OR department_id = 3;
```

Example iii : Using IN to match multiple values

```
SELECT * FROM Employees WHERE department_id IN (1, 3);
```

Example iv : Using BETWEEN to fetch records in a range (dates, numbers, etc.)

```
SELECT * FROM Employees WHERE hire_date BETWEEN '2021-01-01' AND '2022-01-01';
```

Example v: Using IS NULL to find records with NULL values

```
SELECT * FROM Employees WHERE phone_number IS NULL;
```

PAAVAI ENGINEERING COLLEGE (Autonomous)		
Description	Marks	Awarded
Preparation & Conduction	10	
Observation & Results	20	
Record Completion	05	
Viva Voce	05	
Total	40	

Result

Thus, the above program has been executed successfully.

Ex.No :4	CREATING AN EMPLOYEE DATABASE TO SET VARIOUS CONSTRAINTS
Date:	

Aim :

Creating an **Employee Database** with various constraints in a relational database like **Oracle** involves defining tables, setting primary and foreign keys, and adding other types of constraints like CHECK, NOT NULL, UNIQUE, and DEFAULT

Algorithm :

Step 1 : **Create the Employee Table:** This will store employee details such as employee ID, name, position, salary, and date of joining.

Step 2 : **Create the Department Table:** This will store department details to link employees to their respective departments.

Step 3 : **Create Constraints:** Primary keys, foreign keys, not null constraints, check constraints, and others.

i. Create the Database

Let's start by creating the EmployeeDB database:

```
CREATE DATABASE EmployeeDB;
```

ii. Create Tables with Constraints

Now, let's create the Departments and Employees tables, using various constraints.

a. Departments Table

This table will store information about the departments. It will include a PRIMARY KEY constraint for the department identifier.

```
USE EmployeeDB;
```

```
CREATE TABLE Departments (department_id INT PRIMARY KEY, department_name  
VARCHAR(100) NOT NULL, location VARCHAR(100) );
```

b. Employees Table

The Employees table will include several constraints:

Primary Key on employee_id.

Foreign Key on department_id to link the employee to the department.

Not Null on first_name, last_name, and email (i.e., those fields cannot be empty).

Unique on email to prevent duplicate email addresses.

Check on hire_date to ensure employees are hired in the past.

Default for status (e.g., 'active') if not specified.

```
CREATE TABLE Employees (  
    employee_id INT PRIMARY KEY, -- Primary key for unique employee identification  
    first_name VARCHAR(50) NOT NULL, -- First name cannot be NULL  
    last_name VARCHAR(50) NOT NULL, -- Last name cannot be NULL  
    email VARCHAR(100) UNIQUE NOT NULL, -- Email must be unique and not NULL  
    hire_date DATE NOT NULL CHECK (hire_date <= CURDATE()), -- Hire date must be today  
    or earlier  
    status VARCHAR(20) DEFAULT 'Active', -- Default status is 'Active'  
    department_id INT, -- Foreign key linking employee to department
```

```
salary DECIMAL(10, 2) CHECK (salary > 0), -- Salary must be greater than 0
FOREIGN KEY (department_id) REFERENCES Departments(department_id) -- Foreign key
constraint
);
```

c. Adding Constraints After Table Creation

You can also add constraints after creating the table using the ALTER TABLE statement.

Example: Adding a CHECK constraint on salary after the table has been created.

```
ALTER TABLE Employees
ADD CONSTRAINT chk_salary CHECK (salary > 0);
```

d. Inserting Data with Constraints

Now, let's insert data into the Departments and Employees tables. The constraints will be enforced automatically.

```
INSERT INTO Departments (department_id, department_name, location)
VALUES (1, 'HR', 'New York'),
       (2, 'Engineering', 'San Francisco'),
       (3, 'Marketing', 'Chicago');
```

```
INSERT INTO Employees (employee_id, first_name, last_name, email, hire_date, status,
department_id, salary)
VALUES (1, 'Alice', 'Johnson', 'alice.johnson@company.com', '2021-05-01', 'Active', 1, 60000),
       (2, 'Bob', 'Smith', 'bob.smith@company.com', '2020-06-15', 'Active', 2, 80000),
       (3, 'Charlie', 'Brown', 'charlie.brown@company.com', '2022-07-20', 'Inactive', 3, 75000);
```

e. Example Scenarios Using Constraints

Here are some examples where constraints will be enforced.

Example i : Attempting to Insert Duplicate Email

If you try to insert an employee with a duplicate email, you'll get an error because of the UNIQUE constraint.

```
INSERT INTO Employees (employee_id, first_name, last_name, email, hire_date, status,  
department_id, salary)  
VALUES (4, 'David', 'Wilson', 'alice.johnson@company.com', '2023-08-10', 'Active', 2, 70000);
```

Example ii : Attempting to Insert Invalid Salary

If you try to insert an employee with a negative salary, it will fail because of the CHECK constraint on the salary column.

```
INSERT INTO Employees (employee_id, first_name, last_name, email, hire_date, status,  
department_id, salary)  
VALUES (5, 'Eve', 'Davis', 'eve.davis@company.com', '2024-02-01', 'Active', 4, -5000);
```

Example iii : Inserting Invalid Hire Date

If you try to insert an employee with a future hire date, it will fail because of the CHECK constraint on hire_date.

```
INSERT INTO Employees (employee_id, first_name, last_name, email, hire_date, status,  
department_id, salary)  
VALUES (6, 'Frank', 'Miller', 'frank.miller@company.com', '2025-01-01', 'Active', 1, 55000);
```

Example iv : Foreign Key Constraint

If you try to insert an employee with a department_id that does not exist in the Departments table, the insertion will fail because of the FOREIGN KEY constraint.

```
INSERT INTO Employees (employee_id, first_name, last_name, email, hire_date, status,  
department_id, salary)  
VALUES (7, 'Grace', 'Taylor', 'grace.taylor@company.com', '2024-03-01', 'Active', 5, 65000);
```

f.Updating Data with Constraints

You can also use UPDATE statements to modify data, and constraints will continue to be enforced.

```
UPDATE Employees  
SET status = 'Inactive'  
WHERE employee_id = 2;
```

UPDATE Employees

SET salary = -1000

WHERE employee_id = 1; -- This will fail due to the CHECK constraint

g. Viewing Records Based on Constraints

You can easily retrieve data that adheres to the constraints.

Example: Retrieve all active employees with salary greater than 60,000.

SELECT *

FROM Employees

WHERE status = 'Active' AND salary > 60000;

PAAVAI ENGINEERING COLLEGE (Autonomous)		
Description	Marks	Awarded
Preparation & Conduction	10	
Observation & Results	20	
Record Completion	05	
Viva Voce	05	
Total	40	

Result

Thus, the above program has been executed successfully.

Ex.No : 5a	WRITE A PL//SQL BLOCK TO SATISFY SOME CONDITIONS BY ACCEPTING INPUT FROM THE USER
Date:	

Aim :

Create a PL/SQL block, check if a given number is even or odd, and print an appropriate message.

Algorithm :

Step 1 : Get the input from the user

Step 2 : Get the variable to **store user input**.

Step 3 : Check the conditions.

Step 4 : Execute the PL SQL

Step 5 : Stop the Process.

DECLARE

 v_number NUMBER;

BEGIN

 DBMS_OUTPUT.PUT_LINE('Enter a number:');

 v_number := &user_number;

 IF MOD(v_number, 2) = 0 THEN

 DBMS_OUTPUT.PUT_LINE('The number ' || v_number || ' is even.');

 ELSE

 DBMS_OUTPUT.PUT_LINE('The number ' || v_number || ' is odd.');

 END IF;

EXCEPTION

 WHEN OTHERS THEN

 DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);

END;

/

Output :

Enter a number:

The number 7 is odd.

Enter a number:

The number 10 is even.

PAAVAI ENGINEERING COLLEGE (Autonomous)		
Description	Marks	Awarded
Preparation & Conduction	10	
Observation & Results	20	
Record Completion	05	
Viva Voce	05	
Total	40	

Result : Thus, the above program has been executed successfully.

Ex.No :5b	WRITE A PL//SQL BLOCK TO SATISFY SOME CONDITIONS BY ACCEPTING INPUT FROM THE USER
Date:	

Aim :

Create a PL/SQL block, check if the factorial of given number and print an appropriate message.

Algorithm :

Step 1 : Get the input from the user

Step 2 : Get the variable to **store user input**.

Step 3 : Check the conditions.

Step 4 : Execute the PL SQL

Step 5 : Stop the Process.

DECLARE

 v_n NUMBER;

 v_fact NUMBER := 1;

BEGIN

 DBMS_OUTPUT.PUT_LINE('Enter a number to calculate factorials up to:');

 v_n := &user_input;

FOR i IN 1..v_n LOOP

 v_fact := 1; -- Reset factorial value for each iteration

 FOR j IN 1..i LOOP

 v_fact := v_fact * j;

 END LOOP;

 DBMS_OUTPUT.PUT_LINE('Factorial of ' || i || ' is: ' || v_fact);

END LOOP;

EXCEPTION

 WHEN OTHERS THEN

 DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);

END;

/

Output :

Enter a number to calculate factorials up to:

Factorial of 1 is: 1

Factorial of 2 is: 2

Factorial of 3 is: 6

Factorial of 4 is: 24

Factorial of 5 is: 120

PAAVAIENGINEERING COLLEGE (Autonomous)		
Description	Marks	Awarded
Preparation & Conduction	10	
Observation & Results	20	
Record Completion	05	
Viva Voce	05	
Total	40	

Result : Thus, the above program has been executed successfully.

Ex.No : 6	NOSQL Commands using MONGODB
Date:	

Aim :

To write NOSQL QUERIES to understand the concept of Open Source Database Management System such as MongoDB.

Algorithm :

Step 1: Start the MongoDB Server (Mongos).

Step 2: Start the Client (Mongod)

Step 3: Perform the MongoDB Curd Operations such as (Create, Update,Read,Delete).

Syntax: To Create/Select a Collection: USE DATABASE_NAME.

Syntax: To Insert: DB.COLLECTION_NAME.INSERT(DOCUMENT).

Syntax: To Update: DB.COLLECTION_NAME.UPDATE(<FILTER>, <UPDATE>)

Syntax: To Display/Search: DB.COLLECTION_NAME.FIND ()

Syntax: To Delete: DB.COLLECTION_NAME.REMOVE (DELETION_CRITERIA)

Create :

```
> use mca; switched to db mca
```

Insert:

```
> db.mca.studentInfo.insertOne({name: "Surya", age: 23, town: "Salem"});
{
  "acknowledged" : true,
  "insertedId" : ObjectId("62f34f875705a8979214f6cd")
}

> db.mca.studentInfo.insertMany([
  {name: "Arya", age: 23, town: "USA"},
  {name: "Saran", age: 22, town: "Salem"},
  {name: "Ajith", age: 21, town: "Namakkal"},
  {name: "Vijay", age: 40, town: "Chennai"}
]);
{
  "acknowledged" : true, "insertedIds" : [
    ObjectId("62f34fde5705a8979214f6ce"), ObjectId("62f34fde5705a8979214f6cf"),
    ObjectId("62f34fde5705a8979214f6d0"), ObjectId("62f34fde5705a8979214f6d1")
  ]
}
```

Update:

```
> db.mca.studentInfo.updateOne({name: "Vijay"}, {$set:{age: 22}});
```

Delete:

```
> db.mca.studentInfo.deleteOne({name: "Arya"})
> db.mca.studentInfo.remove({name: "Vijay"})
```

Find:

```
> db.mca.studentInfo.find({})
```

Result:

Thus the above MongoDB Queries has been executed successfully

PAAVAI ENGINEERING COLLEGE (Autonomous)		
Description	Marks	Awarded
Preparation & Conduction	10	
Observation & Results	20	
Record Completion	05	
Viva Voce	05	
Total	40	

Ex.No : 7	DATABASE AND TABLE OPERATIONS IN MYSQL
Date :	

Aim:

To write a Query for MySQL Database Creation, Table Creation and Some other Queries Execution.

Algorithm:

Step 1: Start the MYSQL Server.

Step 2: Open MySQL Command Line Client

Step 3: Write and Execute Queries for Database Creation

Step 4: Write and Execute Queries for Table Creation

Step 5: Verify with Show Command to Check whether the Database and Table Created.

Step 6: Do some other Queries Execution like insert, update, delete a record.

Step 7: Close MySQL Command Line Client

Step 8: Stop the MYSQL Server.

```
CREATE DATABASE mca;
```

```
CREATE TABLE mca.student_info(  
id INT PRIMARY KEY, name VARCHAR(30),  
age INT,  
town VARCHAR(40)  
);
```

Insert

```
INSERT INTO mca.student_info VALUES(1, "Surya", 22, "Salem"), (2, "Saran", 21, "Salem"),  
(3, "Hari", 22, "Namakkal"),  
(4, "Vijay", 21, "Salem"),  
(5, "Ajith", 22, "Erode"),  
(6, "Vikram", 21, "Chennai"),  
(7, "Nantha", 22, "Namakkal"),  
(8, "Sathish", 21, "Salem");
```

Update

```
UPDATE mca.student_info SET town="Salem" WHERE id=6;
```

Delete

```
DELETE FROM mca.student_info WHERE id=5;
```

Read

```
SELECT * FROM mca.student_info;
```

```
Terminal - leo@leo-HP-241-G1: ~/code/databases/02_mysql_crud_operation/screenshots
mysql> CREATE DATABASE mca;
Query OK, 1 row affected (0.24 sec)

mysql> CREATE TABLE mca.student_info(
  -> id INT PRIMARY KEY,
  -> name VARCHAR(30),
  -> age INT,
  -> town VARCHAR(40)
  -> );
Query OK, 0 rows affected (1.32 sec)

mysql> INSERT INTO mca.student_info
  -> VALUES(1, "Surya", 22, "Salem"),
  -> (2, "Saran", 21, "Salem"),
  -> (3, "Hari", 22, "Namakkal"),
  -> (4, "Vijay", 21, "Salem"),
  -> (5, "Ajith", 22, "Erode"),
  -> (6, "Vikram", 21, "Chennai"),
  -> (7, "Nantha", 22, "Namakkal"),
  -> (8, "Sathish", 21, "Salem");
Query OK, 8 rows affected (0.15 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> UPDATE mca.student_info SET town="Salem" WHERE id=6;
Query OK, 1 row affected (0.11 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
```

```
Terminal - leo@leo-HP-241-G1: ~/code/databases/02_mysql_crud_operation/screenshots
  -> (6, "Vikram", 21, "Chennai"),
  -> (7, "Nantha", 22, "Namakkal"),
  -> (8, "Sathish", 21, "Salem");
Query OK, 8 rows affected (0.15 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> UPDATE mca.student_info SET town="Salem" WHERE id=6;
Query OK, 1 row affected (0.11 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> DELETE FROM mca.student_info WHERE id=5;
Query OK, 1 row affected (0.19 sec)

mysql> SELECT * FROM mca.student_info;
+----+-----+-----+-----+
| id | name  | age  | town  |
+----+-----+-----+-----+
| 1  | Surya | 22   | Salem |
| 2  | Saran | 21   | Salem |
| 3  | Hari  | 22   | Namakkal |
| 4  | Vijay | 21   | Salem |
| 6  | Vikram | 21   | Salem |
| 7  | Nantha | 22   | Namakkal |
| 8  | Sathish | 21   | Salem |
+----+-----+-----+-----+
7 rows in set (0.10 sec)

mysql>
```

PAAVAI ENGINEERING COLLEGE (Autonomous)		
Description	Marks	Awarded
Preparation & Conduction	10	
Observation & Results	20	
Record Completion	05	
Viva Voce	05	
Total	40	

Result :

Thus the above MYSQL Queries has been executed successfully.

Ex.No : 8	MYSQL REPLICATION – DISTRIBUTED DATABASES
Date :	

Aim:

To apply MYSQL replication technique in Distributed database.

Algorithm :

Step 1: Software to be need to run this Replication Java version 1.8 and above MYSQL Server 8.05 WAMP Server

Step 2: Start the MYSQL Server and WAMP Server.

Step 3: Go to Browser in that type <http://127.0.0.1/> Wamp Server Page will open.

Step 4: In the bottom select tools PHPMyAdmin it will redirect to

<http://127.0.0.1/phpmyadmin/> Step 5: Type User Name as „Root“ and Password „“ and hit enter

Step 6: Create a Database, Table in MYSQL using PHPMYADMIN page.

Step 7: Need 2 MYSQL server, each running as a copy of PHP MyAdmin

Step 8: Your primary Moodle Database is referred to as the “master”, the replicated server will be referred to as the “slave”

Step 9: Take an SQL dump of your master database and restore it on your slave database

Step 10: Now, on your master database log in to PHPMyAdmin and click the Replication Tab

Configuring the Master

The screenshot shows the 'Replication' tab in phpMyAdmin. It contains two sections: 'Master replication' and 'Slave replication'. Both sections have a message stating: 'This server is not configured as [master/slave] in a replication process. Would you like to [configure it](#)?'. The 'configure it' link is highlighted in blue.

MySQL Replication **Step 1**

1. Now click the link to configure this as the master server
2. Select “Ignore all databases, replicate:”
3. Select your Moodle database from the available list (see below)

The screenshot shows the 'Ignore all databases; Replicate:' dropdown menu. Below it, a list of databases is displayed: 'lcm_test_data', 'lcom', 'mahara', 'moodle' (highlighted in blue), 'moodleclean', and 'mysql'. There is an 'Uncheck All' link below the list. Below the list, there is a text box containing the following configuration lines:

```
server-id=129814
log_bin=mysql-bin
log_error=mysql-bin.err
binlog_do_db=moodle
```

Below the text box, there is a message: 'Once you restarted MySQL server, please click on Go button. Afterwards, you should see a message informing you, that this server **is** configured as master'.

Replication **Step 2**

4. Copy and paste the code this screen provides into the very bottom of your MySQL config file (my.ini on Windows) AND add a line that says binlog_format=ROW (this fixes an error when running an external DB enrollment sync with replication)
5. Restart the MySQL services on the master server, leave PHPMyAdmin Open though
6. Once the service has restarted, click on “Go” on the PHPMyAdmin screen.
7. You will be redirected to the Replication screen which now looks like this

Master replication

This server is configured as master in a replication process.

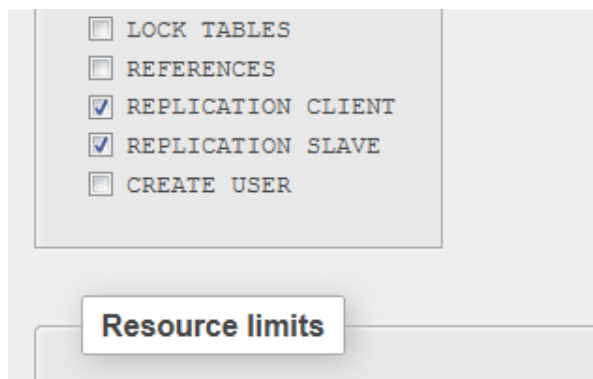
- [Show master status](#)
- [Show connected slaves](#)
- [Add slave replication user](#)

Replication **Step 3**

8. Lastly, we need to create a replication user so click on the link that says “Add slave replicationuser”

9. Create a user and password, set the host to “Any” and click “Go”

10. On the privileges screen ensure the new user has both replication permissions checked and click “Go”



☐	LOCK TABLES
☐	REFERENCES
☒	REPLICATION CLIENT
☒	REPLICATION SLAVE
☐	CREATE USER

Resource limits

Replication **Step 4**

11. That’s all we need to do on the master server, now lets move over to the slave

12. From within PHP MyAdmin on the slave, click the replication tab

13. Then click the link to configure this server as slave replication

14. Copy the line of code that shows your new server id and paste this entire line into the MySQLconfig file on your slave database server

15. Stop and start your MySQL service on the slave server

16. Now in PHP MyAdmin enter the username of the replication user you created in step 13

17. Enter the password and the host (the hostname of the master server or its IP Address)

18. If your default port is not 3306 then change it, chances are it uses the default port

19. Click “Go”

Slave configuration - Change or reconfigure master server

Make sure, you have unique server-id in your configuration file (my.cnf). If not, please add the following line into [mysqld] section:

```
server-id=1372932273
```

User name:

Password :

Host :

Port:

Replication Step 5

20. It then takes you back to the replication screen and appears as though it's not configured but it requires a refresh

21. So refresh the page and you will see it's configured but not running

Slave replication

 Slave SQL Thread not running!

 Slave IO Thread not running!

Server is configured as slave in a replication process. Would you like to:

- [See slave status table](#)
- [Control slave:](#)
- [Error management:](#)
- [Change or reconfigure master server](#)

Replication Step 6

22. Click “Control slave” then click “Full Start”

23. PHPMyAdmin now sits there with a Loading window, after a while this will time out, if this happens or indeed after 5 minutes nothing happens then reload the replication page again (refresh).

24. When the page reloads (either automatically or manually forced by you, you will see no error warnings and a message that says “Server is configured as slave”

25. Now to check it's working, click on the link that says "See slave status table"

Slave replication

Server is configured as slave in a replication process. Would you like to:

- See slave status table

Variable	Value
Slave_IO_State	Waiting for master to send event
Master_Host	spacetest
Master_User	replicationuser
Master_Port	3306

Replication Step 7

26. If everything is working then you will see a message against Slave_IO_State that reads "Waitingfor master to send event"

 **Create database** 

☐	Database	Collation	Master replication	Action
☐	information_schema	utf8_general_ci	 Not replicated	 Check privileges
☐	mysql	latin1_swedish_ci	 Not replicated	 Check privileges
☐	performance_schema	utf8_general_ci	 Not replicated	 Check privileges
☐	rep	latin1_swedish_ci	 Not replicated	 Check privileges
☐	rep1	latin1_swedish_ci	 Not replicated	 Check privileges
☐	sys	utf8_general_ci		
☐	vysya	utf8_general_ci		
Total: 7				

PAAVAIENGINEERING COLLEGE (Autonomous)		
Description	Marks	Awarded
Preparation & Conduction	10	
Observation & Results	20	
Record Completion	05	
Viva Voce	05	
Total	40	

Result :

Thus the above program has been successfully executed.

Ex.No :9	SPATIAL DATA STORAGE AND RETRIEVAL IN MYSQL
Date:	

Aim:

To implement spatial data storage and retrieval in MYSQL.

Algorithm :

Step 1: Start the MYSQL Server.

Step 2: Create a Database.

Step 3: Create a table with spatial data type. Step 4: Insert the spatial values in the table.

Step 5: Display the output using the select command

Step 6: In the output screen Select Form Editor output will be displayed.

```
CREATE DATABASE spatial1;
CREATE TABLE spatial1.restaurants ( name VARCHAR(20),
location GEOMETRY NOT NULL, SPATIAL INDEX(location)
);
```

```
INSERT INTO spatial1.restaurants
VALUES ('Salem', ST_GeomFromText('POINT(11.659365 78.157068)')),
('Erode', ST_GeomFromText('POINT(11.324675 77.723072)')),
('Coimbatore', ST_GeomFromText('POINT(10.993233 76.973483)')),
('Madurai', ST_GeomFromText('POINT(9.883899 78.130947)')),
('Sivakasi', ST_GeomFromText('POINT(9.451873 77.797697)')),
('Tirunelveli', ST_GeomFromText('POINT(8.727173 77.703005)')),
('Rameswaram', ST_GeomFromText('POINT(9.268303 79.243055)')),
('Thanjavur', ST_GeomFromText('POINT(10.782451 79.137895)')),
('Puducherry', ST_GeomFromText('POINT(11.929367 79.809884)));
```

```
SELECT name, ST_AsText(location)FROM spatial1.restaurants
WHERE ST_Distance_Sphere(location, ST_GeomFromText('POINT(11.659365
78.157068)'))<=100*1000
ORDER BY name;
```

```
CREATE DATABASE spatial2
CREATE TABLE restaurants (name VARCHAR(100), location GEOMETRY NOT NULL,
SPATIAL INDEX(location)
);
```

```
INSERT INTO restaurants VALUES ('Restaurant 1', ST_GeomFromText('POINT(-26.66115
40.95858)'));
INSERT INTO restaurants VALUES ('Restaurant 2', ST_GeomFromText('POINT(-26.68685
40.93992)'));
INSERT INTO restaurants VALUES ('Restaurant 3', ST_GeomFromText('POINT(-31.11924
42.39557)'));
SELECT name, AsText(location) FROM restaurants WHERE ST_Distance_Sphere(location,
ST_GeomFromText('POINT(-26.640.9)'))<=10*1000 ORDER BY name;
```

```

Terminal - leo@leo-HP-241-G1: ~
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE spatial1;
Query OK, 1 row affected (0.29 sec)

mysql> CREATE TABLE spatial1.restaurants (
  -> name VARCHAR(20),
  -> location GEOMETRY NOT NULL,
  -> SPATIAL INDEX(location)
  -> );
Query OK, 0 rows affected, 1 warning (3.36 sec)

mysql> INSERT INTO spatial1.restaurants
  -> VALUES('Salem', ST_GeomFromText('POINT(11.659365 78.157068)')),
  -> ('Erode', ST_GeomFromText('POINT(11.324675 77.723072)')),
  -> ('Coimbatore', ST_GeomFromText('POINT(10.993233 76.973483)')),
  -> ('Madurai', ST_GeomFromText('POINT(9.883899 78.130947)')),
  -> ('Sivakasi', ST_GeomFromText('POINT(9.451873 77.797697)')),
  -> ('Tirunelveli', ST_GeomFromText('POINT(8.727173 77.703005)')),
  -> ('Rameswaram', ST_GeomFromText('POINT(9.268303 79.243055)')),
  -> ('Thanjavur', ST_GeomFromText('POINT(10.782451 79.137895)')),
  -> ('Puducherry', ST_GeomFromText('POINT(11.929367 79.809884)'));
Query OK, 9 rows affected (0.92 sec)
Records: 9 Duplicates: 0 Warnings: 0

mysql> SELECT name, ST_AsText(location)
  -> FROM spatial1.restaurants
  -> WHERE ST_Distance_Sphere(location, ST_GeomFromText('POINT(11.659365 78.157068)'))<=100*1000
  -> ORDER BY name;
+-----+-----+
| name      | ST_AsText(location) |
+-----+-----+
| Erode     | POINT(11.324675 77.723072) |
| Madurai   | POINT(9.883899 78.130947) |
| Salem    | POINT(11.659365 78.157068) |
| Sivakasi   | POINT(9.451873 77.797697) |
| Tirunelveli | POINT(8.727173 77.703005) |
+-----+-----+
5 rows in set (0.16 sec)

mysql> █

```

Server: 127.0.0.1 > Database: spatial1 > Table: restaurants

Showing rows 0 - 3 (4 total, Query took 0.0615 seconds.) [name: RESTAURANT 1... - RESTAURANT 2...]

```

SELECT name,AsText(location) FROM restaurants WHERE ST_Distance_Sphere(location, ST_GeomFromText('POINT(-26.6 40.9)'))
<=1000 ORDER BY name;

```

Extra options

name	AsText(location)
Restaurant 1	POINT(-26.66115 40.95858)
Restaurant 1	POINT(-26.66115 40.95858)

Console

```

> SELECT name,AsText(location) FROM restaurants WHERE ST_
>

```

Description	Marks	Awarded
Preparation & Conduction	10	
Observation & Results	20	
Record Completion	05	
Viva Voce	05	
Total	40	

Result :

Thus the above program for spatial data storage and retrieval in MYSQL executed successfully.

Ex.No : 10	TEMPORAL DATA STORAGE AND RETRIEVAL IN MYSQL
Date:	

Aim:

To Create a Temporal data storage and retrieve that data in mysql.

Algorithm :

Step 1: Start the MYSQL Server.

Step 2: Create a Database.

Step 3: Create Two Tables with the name of Customers and Orders.

Step 4: Insert Some record in that tables.

Step 5: Create a Temporary Table by using the object we can create it

Step 6: Show the temporary table.

CREATE DATABASE temporal1;

```
CREATE TABLE temporal1.student_details ( Student_Id INT  
AUTO_INCREMENT,First_name VARCHAR (100) NOT NULL,Last_name  
VARCHAR (100) NOT NULL, Date_Of_Birth DATE NOT NULL, Class  
VARCHAR (10) NOT NULL,Contact_DetailsBIGINT NOT NULL,PRIMARY  
KEY(Student_Id ));
```

```
INSERT INTO temporal1.student_details(First_name , Last_name ,  
Date_Of_Birth ,Class, Contact_Details)VALUES('John', 'stark', '2022-07-16',  
'MCA', 1234567890),('Arya', 'stark','2003-06-10', 'BE' , 1214657890);
```

```
SELECT * FROM temporal1.student_details;
```

```
mysql> CREATE DATABASE temporal1;  
Query OK, 1 row affected (0.23 sec)  
  
mysql> CREATE TABLE temporal1.student_details (  
-> Student_Id INT AUTO_INCREMENT,  
-> First_name VARCHAR (100) NOT NULL,  
-> Last_name VARCHAR (100) NOT NULL,  
-> Date_Of_Birth DATE NOT NULL,  
-> Class VARCHAR (10) NOT NULL,  
-> Contact_Details BIGINT NOT NULL,  
-> PRIMARY KEY(Student_Id )  
-> );  
Query OK, 0 rows affected (1.60 sec)  
  
mysql> INSERT INTO temporal1.student_details(First_name , Last_name , Date_Of_Birth , Class, Contact_Details)  
-> VALUES('John', 'stark', '2022-07-16', 'MCA', 1234567890),  
-> ('Arya', 'stark', '2003-06-10', 'BE' , 1214657890);  
Query OK, 2 rows affected (0.20 sec)  
Records: 2 Duplicates: 0 Warnings: 0  
  
mysql> SELECT * FROM temporal1.student_details;  
+-----+-----+-----+-----+-----+-----+  
| Student_Id | First_name | Last_name | Date_Of_Birth | Class | Contact_Details |  
+-----+-----+-----+-----+-----+-----+  
| 1 | John | stark | 2022-07-16 | MCA | 1234567890 |  
| 2 | Arya | stark | 2003-06-10 | BE | 1214657890 |  
+-----+-----+-----+-----+-----+-----+
```

PAAVAI ENGINEERING COLLEGE (Autonomous)		
Description	Marks	Awarded
Preparation & Conduction	10	
Observation & Results	20	
Record Completion	05	
Viva Voce	05	
Total	40	

Result:

Thus the above program Temporary data storage and retrieval as been executed successfully